

Notes Pack: Section 5.1 fi-why — Why the Item Side First

Placed protocol entries (in section order), then their one-hop linked entries. Generated 2026-08-20; working material, not thesis text.

[placed] 2026-07-11_2350_interactions-explained-by-item-features-taste-revealed

Why an interaction is explainable by item features but not by user features — taste is revealed, not stated

(Verbatim-capture entry from the dc01 SC.1b gate detour — thesis-motivation material; the user asked for exactly this argument to be protocolled. It grounds the item-side-first strategy and the C1 lineage outline.)

The core argument

An interaction happens when **item content meets user taste** — the bilinear score literally has that shape: $\text{taste-vector} \cdot \text{content-vector}$. The two causal ingredients have opposite epistemic status:

- **Item content is observable and recorded.** What the garment is — the catalog writes it down in 426 words (departments, colours, patterns...). Composing items from features (dc01) grounds that side in the actual ingredient of the interaction.
- **User taste is latent and unrecorded.** Users don't state their taste; mostly they *couldn't* state it if asked — nobody can articulate their own taste function. Stated personal attributes (age, club membership, newsletter frequency) are a very loose proxy: enough signal for cold-user popularity fallbacks (what the Kaggle field used them for), nowhere near enough to *ground an explanation in*. Grounding user prototypes in demographics would ground them in a proxy so loose the grounding would be a lie ("recommended because you are 34 and get the newsletter" — R7-wrong, privacy-adjacent).

The refinement that completes it: taste is not *unobtainable* — it is unobtainable as a **stated feature**, but richly **revealed** through behavior (revealed preference). The purchase history is a high-fidelity record of taste, and capturing it is exactly what CF already does: the user embedding IS learned, revealed taste. The user side's problem was never *capture* — it is that revealed taste arrives **without labels**: a latent vector with no vocabulary attached. Therefore: **don't hunt for taste in user attributes; express revealed taste in item vocabulary** — describe a taste community by what its members buy.

The four-component honesty inventory (dc01 as it stands)

The UI score has four contributing values; in dc01, their explainability splits: **t*** (item's prototype activations) — grounded, feature-decomposable; **û** (user's affinities over named item prototypes) — derived-legible for free; **î** (item's appeal profile over user communities) — feature-attributable (linear-half read-out) but pointing at ungrounded units; **u*** (user's community memberships) — ungrounded, post-hoc-named as in the host. User principle, adopted: *whatever contributes to the score cannot be left unexplained* — the current honest answer is disclosure (S0.2's binding score-share reporting); the lineage outline (companion entry [[2026-07-11_2351_thesis-outline-c1-lineage]]) is the plan for closing it for real.

The supporting data asymmetry (from the same discussion)

In pure CF, *both* embedding spaces are behavior summaries — an item’s CF embedding encodes *who bought it*, not what it is; the item side’s legibility was always **borrowed** from the catalog’s external vocabulary. dc01 moves that borrowing inside the model. The user side has no equivalent rich vocabulary to move inside (H&M: ~3 thin fields, ~15 usable values, postal code hashed/identity-like) — hence item-side-first is not a preference but a data-forced strategy, and the user side’s eventual grounding must route through item vocabulary (history-composed representations / projected community profiles).

[[decisions]] [[explanations]] [[phase-4]]

[**linked**] 2026-07-11_2351_thesis-outline-c1-lineage

Thesis outline: the Candidate-1 lineage (user proposal, dc01 SC.1b gate detour)

(Verbatim-capture entry — the user’s proposed thesis-flow for part one, with the agreed cautions. Presentation order for the thesis; NOT a license to skip the design-candidate + scrutiny machinery for each new stage.)

The lineage (user’s outline, kept close to literal)

1. **Present Candidate 1 completely and fully**, WITHOUT the half-opaque part resolved — immediately transparent about it (the four-component inventory: t^* grounded, $\hat{u}$ derived-legible, $\hat{t}$ attributable-but-pointing-at-ungrounded-units, u^* ungrounded; S0.2 score-share disclosure).
2. **Explain why** — the protocolled argument that an interaction is far better explained by item features than by user features: taste has to be *discovered/revealed*, not stated; demographics are a too-loose proxy ([[2026-07-11_2350_interactions-explained-by-item-features-taste-revealed]]).
3. **Demonstrate the explanation as it stands**: our grounded explanation + the opaque user component shown honestly.
4. **Introduce the route-1 component** (user prototypes projected through the tie into named item-prototype space) — show how it would look, AND what its problems are (enabled-not-enforced, extrapolation caveat: the projection matrix was trained on user embeddings, not prototypes).
5. **Present a candidate matching route 2** (history-composed user representation — user = aggregate of purchased items’ feature words) — quite distinct; conditional on implementing it with promising results; brings the new explanation dimension (per-purchase attribution) + its own item component.
6. **Merge the two, following the UI-ProtoMF paper’s own structure** (item-side $\rightarrow$ user-side $\rightarrow$ merged double-tie) — the thesis extends ProtoMF the way ProtoMF built itself.
7. **Route 2 begs the cold-user question** (new user = empty history = empty aggregate — the cold-start keyword rings). This motivates **candidate prime**: user attributes additionally enter the user side and its prototypes (still future land). Expected honest result: *mostly meaningless, occasionally chips into the explanation — and in new-user scenarios even improves accuracy*. Demographics enter last, weakest, and only where nothing else exists — the placement the whole taste argument demands. (Math note: the aggregate degrades gracefully — user = Σ history-features + Σ attribute rows (+ ID row); a cold user just has an empty first sum — LightFM’s cold story one level up.)
8. **Close the lineage with speculated further improvements**, e.g. a CNN bringing images in directly — image recognition tied to the recommendation-loss optimization,

no separate feature-extraction phase (one more row type in the sum; speculated, not promised).

Fallback/parallel: if stuck anywhere, borrow ideas from candidates 2/3/4 or continue their development — potentially deriving similar lineages from them; parallel literature reading feeds fresh ideas in. But this lineage comes first.

Structure of the thesis: this lineage = part one; the **“Interpretability quantified” section comes strictly after** (facet-A instrumentation: profile entropy/sharpness, feature-explained fraction, purity, determinacy annotations, Rashomon-set tuning).

Commitment and discipline

- **The lineage is the committed direction, not a scope risk** (*user, explicitly, same gate — overriding Claude’s initial “lineage with exits” hedge*): “if that works out I will push it and I will get there, because it’s just so convincing and I am just so passionate about that.” The exits (thesis defensible after C1 + quantified interpretability; candidates 2–4 as fallback/parallel lineages) exist as a safety net, NOT as the plan. Sessions should treat the lineage as the destination and pace toward it, not question its feasibility by default.
- **Process discipline (unchanged):** route 2, the merge, and prime each pass through the design-candidate protocol (prior-art probe, math sanity, devil’s advocate) and scrutiny before entering the lineage as results. Presentation order $\neq$ process order. Passion sets the destination; the protocols keep every step on it trustworthy.
- Current position: mid-SC(dc01) scrutiny; the lineage is where we are heading.

[[decisions]] [[phase-4]] [[explanations]]